

An introduction to Lean

Tetsuya Ishiu

Department of Mathematics
Miami University

Monday, September 14, 2026

Formal Proof

It was already known that machine-verifiable proofs were technically possible in the early 20th century.

- Proof Theory is a branch of mathematical logic, in which we formalize proofs and study them.
- An early result of Proof Theory is that every mathematical statement can be considered as a sequence of symbols, and proofs can be considered as a sequence of such statements that satisfy certain rules.
- Whether a sequence satisfies the certain rules or not can be mechanically checked. Namely, we can make a program that checks it.
- These things are covered in MTH 483/583 Introduction to Mathematical Logic.

Formal Proof

It was already known that machine-verifiable proofs were technically possible in the early 20th century.

- Proof Theory is a branch of mathematical logic, in which we formalize proofs and study them.
- An early result of Proof Theory is that every mathematical statement can be considered as a sequence of symbols, and proofs can be considered as a sequence of such statements that satisfy certain rules.
- Whether a sequence satisfies the certain rules or not can be mechanically checked. Namely, we can make a program that checks it.
- These things are covered in MTH 483/583 Introduction to Mathematical Logic.

Formal Proof

It was already known that machine-verifiable proofs were technically possible in the early 20th century.

- Proof Theory is a branch of mathematical logic, in which we formalize proofs and study them.
- An early result of Proof Theory is that every mathematical statement can be considered as a sequence of symbols, and proofs can be considered as a sequence of such statements that satisfy certain rules.
- Whether a sequence satisfies the certain rules or not can be mechanically checked. Namely, we can make a program that checks it.
- These things are covered in MTH 483/583 Introduction to Mathematical Logic.

Formal Proof

It was already known that machine-verifiable proofs were technically possible in the early 20th century.

- Proof Theory is a branch of mathematical logic, in which we formalize proofs and study them.
- An early result of Proof Theory is that every mathematical statement can be considered as a sequence of symbols, and proofs can be considered as a sequence of such statements that satisfy certain rules.
- Whether a sequence satisfies the certain rules or not can be mechanically checked. Namely, we can make a program that checks it.
- These things are covered in MTH 483/583 Introduction to Mathematical Logic.

Formal Proof

It was already known that machine-verifiable proofs were technically possible in the early 20th century.

- Proof Theory is a branch of mathematical logic, in which we formalize proofs and study them.
- An early result of Proof Theory is that every mathematical statement can be considered as a sequence of symbols, and proofs can be considered as a sequence of such statements that satisfy certain rules.
- Whether a sequence satisfies the certain rules or not can be mechanically checked. Namely, we can make a program that checks it.
- These things are covered in MTH 483/583 Introduction to Mathematical Logic.

Type Theory

- However, most of proof assistants were not built on this idea.
- Instead, they are based on Type Theory.
- Type Theory was first introduced by Bertrand Russell to avoid Russell's Paradox.
 - $R \in R$ should not be asked because it is not well-typed.
- The famous book 'Principia Mathematica' by Alfred North Whitehead and Bertrand Russell was written based on Type Theory.
- Many people argue that Type Theory is a better foundation of mathematics than Set Theory.
- Type Theory is now very important in Computer Science.

Type Theory

- However, most of proof assistants were not built on this idea.
- Instead, they are based on Type Theory.
- Type Theory was first introduced by Bertrand Russell to avoid Russell's Paradox.
 - $R \in R$ should not be asked because it is not well-typed.
- The famous book 'Principia Mathematica' by Alfred North Whitehead and Bertrand Russell was written based on Type Theory.
- Many people argue that Type Theory is a better foundation of mathematics than Set Theory.
- Type Theory is now very important in Computer Science.

Type Theory

- However, most of proof assistants were not built on this idea.
- Instead, they are based on Type Theory.
- Type Theory was first introduced by Bertrand Russell to avoid Russell's Paradox.
 - $R \in R$ should not be asked because it is not well-typed.
- The famous book 'Principia Mathematica' by Alfred North Whitehead and Bertrand Russell was written based on Type Theory.
- Many people argue that Type Theory is a better foundation of mathematics than Set Theory.
- Type Theory is now very important in Computer Science.

Type Theory

- However, most of proof assistants were not built on this idea.
- Instead, they are based on Type Theory.
- Type Theory was first introduced by Bertrand Russell to avoid Russell's Paradox.
 - $R \in R$ should not be asked because it is not well-typed.
- The famous book 'Principia Mathematica' by Alfred North Whitehead and Bertrand Russell was written based on Type Theory.
- Many people argue that Type Theory is a better foundation of mathematics than Set Theory.
- Type Theory is now very important in Computer Science.

Type Theory

- However, most of proof assistants were not built on this idea.
- Instead, they are based on Type Theory.
- Type Theory was first introduced by Bertrand Russell to avoid Russell's Paradox.
 - $R \in R$ should not be asked because it is not well-typed.
- The famous book 'Principia Mathematica' by Alfred North Whitehead and Bertrand Russell was written based on Type Theory.
- Many people argue that Type Theory is a better foundation of mathematics than Set Theory.
- Type Theory is now very important in Computer Science.

Type Theory

- However, most of proof assistants were not built on this idea.
- Instead, they are based on Type Theory.
- Type Theory was first introduced by Bertrand Russell to avoid Russell's Paradox.
 - $R \in R$ should not be asked because it is not well-typed.
- The famous book 'Principia Mathematica' by Alfred North Whitehead and Bertrand Russell was written based on Type Theory.
- Many people argue that Type Theory is a better foundation of mathematics than Set Theory.
- Type Theory is now very important in Computer Science.

Type Theory

- However, most of proof assistants were not built on this idea.
- Instead, they are based on Type Theory.
- Type Theory was first introduced by Bertrand Russell to avoid Russell's Paradox.
 - $R \in R$ should not be asked because it is not well-typed.
- The famous book 'Principia Mathematica' by Alfred North Whitehead and Bertrand Russell was written based on Type Theory.
- Many people argue that Type Theory is a better foundation of mathematics than Set Theory.
- Type Theory is now very important in Computer Science.

Curry-Howard Correspondence

- The Curry-Howard Correspondence is a correspondence between proofs and programs.
- It insists that a mathematical statement is a type, and a proof is an object in that type.
- If we set up the notions appropriately, we can prove it.
- So, we can make an application that is an interpreter of a programming language with such a type system and a proof checker at the same time.
- This is how modern proof assistants are designed.
- Remark: This ‘proof’ usually uses intuitionistic logic, which does not admit the law of excluded middle.

Curry-Howard Correspondence

- The Curry-Howard Correspondence is a correspondence between proofs and programs.
- It insists that a mathematical statement is a type, and a proof is an object in that type.
- If we set up the notions appropriately, we can prove it.
- So, we can make an application that is an interpreter of a programming language with such a type system and a proof checker at the same time.
- This is how modern proof assistants are designed.
- Remark: This ‘proof’ usually uses intuitionistic logic, which does not admit the law of excluded middle.

Curry-Howard Correspondence

- The Curry-Howard Correspondence is a correspondence between proofs and programs.
- It insists that a mathematical statement is a type, and a proof is an object in that type.
- If we set up the notions appropriately, we can prove it.
- So, we can make an application that is an interpreter of a programming language with such a type system and a proof checker at the same time.
- This is how modern proof assistants are designed.
- Remark: This ‘proof’ usually uses intuitionistic logic, which does not admit the law of excluded middle.

Curry-Howard Correspondence

- The Curry-Howard Correspondence is a correspondence between proofs and programs.
- It insists that a mathematical statement is a type, and a proof is an object in that type.
- If we set up the notions appropriately, we can prove it.
- So, we can make an application that is an interpreter of a programming language with such a type system and a proof checker at the same time.
- This is how modern proof assistants are designed.
- Remark: This ‘proof’ usually uses intuitionistic logic, which does not admit the law of excluded middle.

Curry-Howard Correspondence

- The Curry-Howard Correspondence is a correspondence between proofs and programs.
- It insists that a mathematical statement is a type, and a proof is an object in that type.
- If we set up the notions appropriately, we can prove it.
- So, we can make an application that is an interpreter of a programming language with such a type system and a proof checker at the same time.
- This is how modern proof assistants are designed.
- Remark: This ‘proof’ usually uses intuitionistic logic, which does not admit the law of excluded middle.

Curry-Howard Correspondence

- The Curry-Howard Correspondence is a correspondence between proofs and programs.
- It insists that a mathematical statement is a type, and a proof is an object in that type.
- If we set up the notions appropriately, we can prove it.
- So, we can make an application that is an interpreter of a programming language with such a type system and a proof checker at the same time.
- This is how modern proof assistants are designed.
- Remark: This ‘proof’ usually uses intuitionistic logic, which does not admit the law of excluded middle.

Rough justification

Let $x : \tau$ denotes that x is an object of type τ . As I said, it can be interpreted as x is a proof of the statement τ .

- The proof of $\tau_1 \wedge \tau_2$ (τ_1 and τ_2) is an ordered pair $\langle x_1, x_2 \rangle$ where $x_1 : \tau_1$ and $x_2 : \tau_2$.
- The proof of $\tau_1 \vee \tau_2$ (τ_1 or τ_2) is an ordered pair $\langle 1, x_1 \rangle$ where $x_1 : \tau_1$ or $\langle 2, x_2 \rangle$ where $x_2 : \tau_2$.
- The proof of $\tau_1 \rightarrow \tau_2$ (τ_1 implies τ_2) is a function f such that for any $x_1 : \tau_1$, $f(x_1) : \tau_2$.
- What is the proof of $\forall x : \nu, \tau(x)$? It is a function f such that for any $x : \nu$, $f(x) : \tau(x)$. Such a type is called a *dependent type*.

Rough justification

Let $x : \tau$ denotes that x is an object of type τ . As I said, it can be interpreted as x is a proof of the statement τ .

- The proof of $\tau_1 \wedge \tau_2$ (τ_1 and τ_2) is an ordered pair $\langle x_1, x_2 \rangle$ where $x_1 : \tau_1$ and $x_2 : \tau_2$.
- The proof of $\tau_1 \vee \tau_2$ (τ_1 or τ_2) is an ordered pair $\langle 1, x_1 \rangle$ where $x_1 : \tau_1$ or $\langle 2, x_2 \rangle$ where $x_2 : \tau_2$.
- The proof of $\tau_1 \rightarrow \tau_2$ (τ_1 implies τ_2) is a function f such that for any $x_1 : \tau_1$, $f(x_1) : \tau_2$.
- What is the proof of $\forall x : \nu, \tau(x)$? It is a function f such that for any $x : \nu$, $f(x) : \tau(x)$. Such a type is called a *dependent type*.

Rough justification

Let $x : \tau$ denotes that x is an object of type τ . As I said, it can be interpreted as x is a proof of the statement τ .

- The proof of $\tau_1 \wedge \tau_2$ (τ_1 and τ_2) is an ordered pair $\langle x_1, x_2 \rangle$ where $x_1 : \tau_1$ and $x_2 : \tau_2$.
- The proof of $\tau_1 \vee \tau_2$ (τ_1 or τ_2) is an ordered pair $\langle 1, x_1 \rangle$ where $x_1 : \tau_1$ or $\langle 2, x_2 \rangle$ where $x_2 : \tau_2$.
- The proof of $\tau_1 \rightarrow \tau_2$ (τ_1 implies τ_2) is a function f such that for any $x_1 : \tau_1$, $f(x_1) : \tau_2$.
- What is the proof of $\forall x : \nu, \tau(x)$? It is a function f such that for any $x : \nu$, $f(x) : \tau(x)$. Such a type is called a *dependent type*.

Rough justification

Let $x : \tau$ denotes that x is an object of type τ . As I said, it can be interpreted as x is a proof of the statement τ .

- The proof of $\tau_1 \wedge \tau_2$ (τ_1 and τ_2) is an ordered pair $\langle x_1, x_2 \rangle$ where $x_1 : \tau_1$ and $x_2 : \tau_2$.
- The proof of $\tau_1 \vee \tau_2$ (τ_1 or τ_2) is an ordered pair $\langle 1, x_1 \rangle$ where $x_1 : \tau_1$ or $\langle 2, x_2 \rangle$ where $x_2 : \tau_2$.
- The proof of $\tau_1 \rightarrow \tau_2$ (τ_1 implies τ_2) is a function f such that for any $x_1 : \tau_1$, $f(x_1) : \tau_2$.
- What is the proof of $\forall x : \nu, \tau(x)$? It is a function f such that for any $x : \nu$, $f(x) : \tau(x)$. Such a type is called a *dependent type*.

Rough justification

Let $x : \tau$ denotes that x is an object of type τ . As I said, it can be interpreted as x is a proof of the statement τ .

- The proof of $\tau_1 \wedge \tau_2$ (τ_1 and τ_2) is an ordered pair $\langle x_1, x_2 \rangle$ where $x_1 : \tau_1$ and $x_2 : \tau_2$.
- The proof of $\tau_1 \vee \tau_2$ (τ_1 or τ_2) is an ordered pair $\langle 1, x_1 \rangle$ where $x_1 : \tau_1$ or $\langle 2, x_2 \rangle$ where $x_2 : \tau_2$.
- The proof of $\tau_1 \rightarrow \tau_2$ (τ_1 implies τ_2) is a function f such that for any $x_1 : \tau_1$, $f(x_1) : \tau_2$.
- What is the proof of $\forall x : \nu, \tau(x)$? It is a function f such that for any $x : \nu$, $f(x) : \tau(x)$. Such a type is called a *dependent type*.

Rough justification

Let $x : \tau$ denotes that x is an object of type τ . As I said, it can be interpreted as x is a proof of the statement τ .

- The proof of $\tau_1 \wedge \tau_2$ (τ_1 and τ_2) is an ordered pair $\langle x_1, x_2 \rangle$ where $x_1 : \tau_1$ and $x_2 : \tau_2$.
- The proof of $\tau_1 \vee \tau_2$ (τ_1 or τ_2) is an ordered pair $\langle 1, x_1 \rangle$ where $x_1 : \tau_1$ or $\langle 2, x_2 \rangle$ where $x_2 : \tau_2$.
- The proof of $\tau_1 \rightarrow \tau_2$ (τ_1 implies τ_2) is a function f such that for any $x_1 : \tau_1$, $f(x_1) : \tau_2$.
- What is the proof of $\forall x : \nu, \tau(x)$? It is a function f such that for any $x : \nu$, $f(x) : \tau(x)$. Such a type is called a *dependent type*.

Rough justification

Let $x : \tau$ denotes that x is an object of type τ . As I said, it can be interpreted as x is a proof of the statement τ .

- The proof of $\tau_1 \wedge \tau_2$ (τ_1 and τ_2) is an ordered pair $\langle x_1, x_2 \rangle$ where $x_1 : \tau_1$ and $x_2 : \tau_2$.
- The proof of $\tau_1 \vee \tau_2$ (τ_1 or τ_2) is an ordered pair $\langle 1, x_1 \rangle$ where $x_1 : \tau_1$ or $\langle 2, x_2 \rangle$ where $x_2 : \tau_2$.
- The proof of $\tau_1 \rightarrow \tau_2$ (τ_1 implies τ_2) is a function f such that for any $x_1 : \tau_1$, $f(x_1) : \tau_2$.
- What is the proof of $\forall x : \nu, \tau(x)$? It is a function f such that for any $x : \nu$, $f(x) : \tau(x)$. Such a type is called a *dependent type*.

Famous theorems that were formalized

Year	Theorem	Proof System	Formalizer
1986	First Incompleteness	Boyer-Moore	Shankar
1990	Quadratic Reciprocity	Boyer-Moore	Russinoff
1996	Fundamental Thm. of Calculus	HOL Light	Harrison
2000	Fundamental Thm. of Algebra	Mizar	Milewski
2000	Fundamental Thm. of Algebra	Coq	Geuvers et al.
2004	Four-Color Thm.	Coq	Gonthier
2004	Prime Number Thm.	Isabelle	Avigad et al.
2005	Jordan Curve Thm.	HOL Light	Hales
2005	Brouwer Fixed Point	HOL Light	Harrison
2007	Cauchy Residue	HOL Light	Harrison
2008	Prime Number	HOL Light	Harrison
2014	Kepler Conjecture	Isabelle HOL light	Flyspeck project

What happened this year

We see an explosion of the reports of formalizations this year.

Fermat's Last Theorem	Lean	Claude (Anthropic)
Sphere packing in dimension 8	Lean	Gauss (Math Inc.)
The strong Prime Number Theorem	Lean	Gauss (Math Inc.)
Erdős's unit-distance conjecture (disproved)	Lean	OpenAI
Several Erdős problems	Lean	DeepMind(Google)

As you can see, AI models are dominating now.

Lean

- Lean is a relatively new proof assistant, which was launched in 2013.
- It has quickly become popular among mathematicians.
- It is based on a particular type theory called ‘Calculus of Inductive Constructions’.
 - Another proof assistant based on this type theory is Rocq (formerly known as Coq).
 - It was probably the most popular proof assistant among mathematicians before Lean.

Lean

- Lean is a relatively new proof assistant, which was launched in 2013.
- It has quickly become popular among mathematicians.
- It is based on a particular type theory called 'Calculus of Inductive Constructions'.
 - Another proof assistant based on this type theory is Rocq (formerly known as Coq).
 - It was probably the most popular proof assistant among mathematicians before Lean.

Lean

- Lean is a relatively new proof assistant, which was launched in 2013.
- It has quickly become popular among mathematicians.
- It is based on a particular type theory called ‘Calculus of Inductive Constructions’.
 - Another proof assistant based on this type theory is Rocq (formerly known as Coq).
 - It was probably the most popular proof assistant among mathematicians before Lean.

Lean

- Lean is a relatively new proof assistant, which was launched in 2013.
- It has quickly become popular among mathematicians.
- It is based on a particular type theory called ‘Calculus of Inductive Constructions’.
 - Another proof assistant based on this type theory is Rocq (formerly known as Coq).
 - It was probably the most popular proof assistant among mathematicians before Lean.

Lean

- Lean is a relatively new proof assistant, which was launched in 2013.
- It has quickly become popular among mathematicians.
- It is based on a particular type theory called ‘Calculus of Inductive Constructions’.
 - Another proof assistant based on this type theory is Rocq (formerly known as Coq).
 - It was probably the most popular proof assistant among mathematicians before Lean.

How to get started

- Natural Number Game is an online game to learn Lean. By solving each stage, you will learn how to write proofs.
- Lean Web provides an installation-free online environment that can be used from anywhere. When you want to test a short code, this works very well.
- GitHub Codespaces is a cloud development environment provided by GitHub. It is relatively easy to build an environment for Lean.
- If you want to install it locally for greater flexibility, you may want to install VS Code and install Lean 4 extension.

How to make an environment in GitHub Codespaces for Lean

Let me demonstrate how to build a codespace for Lean. I will make a video describing this process, as there are too many small steps.

- 1 Log into GitHub. If you do not have an account, create one.
- 2 Use `Lean_template` as a template to make a repository.
- 3 Go to GitHub Codespaces
- 4 Make a new codespace based on the repository. The Machine Type must be *4-core*.

- It automatically installs Lean by the set-up of

(Continue to the next page)

How to make an environment in GitHub Codespaces for Lean (Cont.)

- 5 Make a new Lean project Using Mathlib (I assume the project name is 'test'). You need to wait for a few minutes. By clicking on 'Click for details', you can see the progress.
- 6 Open 'Test/Basic.lean'. This is a minimal Lean code.
- 7 If you want to prove something in mathematics, write `import Mathlib` on the first line. It will import Mathlib4, the library for mathematics.
 - It takes at least 15 minutes without visible progress. Just wait.
- 8 Now, you are ready to write a formal proof in mathematics.

Introductory part

I would like to split an hour into two 30-minute parts, introductory part and intermediate part. In the introductory part:

- We use Natural Number Game as a text.
 - After finishing Tutorial World and Addition World, I would like you to go as follows: Implication World→Multiplication World→Advanced Addition World→ $\leq$ World→Advanced Multiplication World.
- I will designate a presenter for the next meeting, and they will present their progress.
 - You can ask me or others any questions.
 - It is OK to say 'I cannot understand this' so that we can work on it together.
 - Every participant is encouraged to solve exercises.

Intermediate part

In the intermediate part:

- I will ask for a volunteer speaker or assign someone if there is no volunteer.
- The presenter can present any topic about Lean.